Terminus Data Platform Strategy

1. Current state of Terminus data platform

1.1. Storage

There are two ways to store data in Terminus:

a. BigQuery

We use BigQuery to persist raw data and curated data. Raw data are received from various sources, including vendors, legacy DBs, operational DBs, and business teams. Curated data refer to data that are carefully prepared for use in analytics, reporting, or decision-making processes.

Data deletion are only done with specific requirements (e.g., retention policy).

b. GCS bucket

Based on our regulatory requirements, we cannot store un-encrypted data in GCS bucket. However, our encryption solution is still not in place. So, we only use GCS bucket as temporary storage in our data pipelines.

Data clean up is done in the data pipelines or by the retention policies on the GCS buckets. The default retention period is 7 days.

1.2. Compute

Compute resources include BigQuery, Spark, and Cloud functions.

a. BigQuery

For compute, BigQuery is our first choice for both light-weight computes and heavy-duty computes. BigQuery provides both data storage and powerful compute capabilities. Our data are primarily stored in BigQuery. And it is also easy to add external data sources (e.g., CSV, parquet files) to BigQuery. So, unless necessary, we want to avoid moving data out of BigQuery.

b. Dataproc (Spark)

Apache Spark is a fast, open-source, distributed data processing framework designed for big data applications. Spark jobs are initially created for ingesting large files from vendors (EBCDIC files from TSYS, and plain text files from other vendors). The biggest file we receive is the account master file which is over 200 GB. We need to import this file to BigQuery on daily basis. The main library used for file ingestion is Cobrix.

> There are Spark jobs for creating vendor files too. These jobs are actually not appropriate use cases of Spark. Because Spark is a distributed compute framework, while file generation usually requires sequential processing.

Later, Spark jobs are created to ingest data from legacy databases (ODS, DB303, DB053, and so on) and also insert data into databases (Oracle and Postgres). These Spark jobs communicate with DBs via JDBC connections.

Recently, Spark jobs are used to aggregate quality scan results to build data quality reports.

c. Cloud functions

Cloud function is a serverless compute service. We typically use cloud functions to implement event based data pipelines.

For use cases where sequential processing is needed, cloud function is the choice too. The 1st generation cloud function has many limitations like memory and timeouts. So, they were not widely used in computation tasks.

The 2nd generation cloud function is built on top of cloud run, and is an event driven version of cloud run. It provides much better scalability and availability than 1st generation cloud function.

> Note: We don't use Composer (Airflow) as compute. Airflow is designed for orchestration of flows (or data pipelines). Putting compute in Airflow will cause resource (CPU, memory, I/O etc.) competition and eventually affect the stability and availability of our data pipelines.

1.3. Discovery

Dataplex provides great tools for discovery (e.g., Dataplex universal catalog). We are in the process of enabling Dataplex universal catalog.

1.4. Resource provision

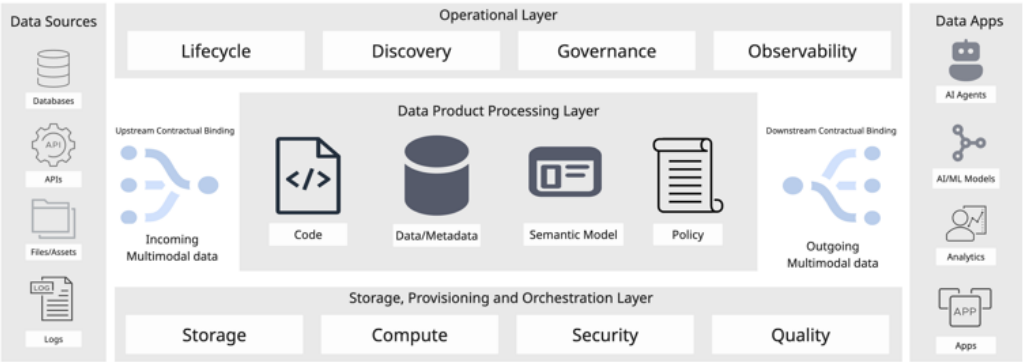
All our GCP resources are provisioned using infrastructure as code (IaC). We use terrform and terragrunt in our infrastructure repositories.

1.5. Observability

We implemented log based monitoring and alerts for our Data pipelines. Alerts are posted in slack channels.

Grafana dashboards have also been built for monitoring critical resources (e.g., BigQuery slot usage) and costs.

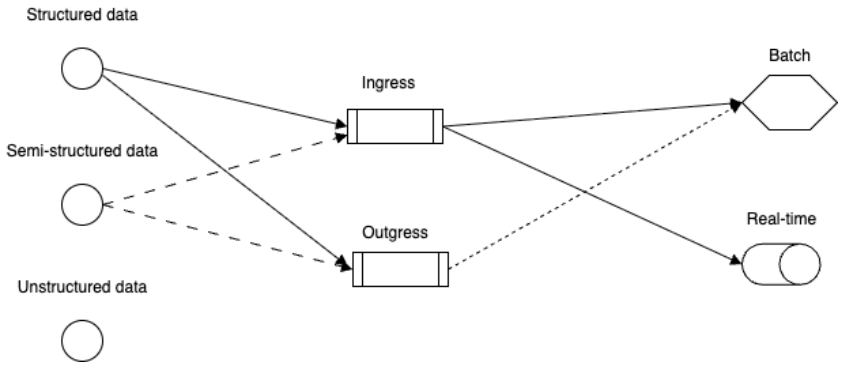
2. Proposed data platform strategy



The current `Terminus Data Platform` is predominantly associated with 'bare metal' infrastructure, specifically the three GCP zones and the associated tools. While we have developed generic classes and DAGs for common use cases, we currently lack a standardized and well-defined set of tools. Consequently, outcome teams are continually adding custom solutions for various use cases.

2.1. Full spectrum support

The future state of Terminus is illustrated in the diagram above. In addition to structured data, we aim to support unstructured and semi-structured data. It is essential to establish approaches for ingesting, processing, and storing both unstructured and semi-structured data.



2.2. Standardized practices

2.2.1. Common naming conventions

Basic Naming Guidelines in GCP (Terminus)

2.2.2. Common ways of define and maintain (evolve) data schemas

BigQuery tables are created in different ways currently.

- Some are created using terraform with schema in JSON files.
- Some are derived from schemas of source data, including copybooks of TSYS files and DB schemas from relation databases.
- There are also schemas defined in YAML files for loading CSV files to BigQuery.

We should have a standard way of defining BigQuery schemas, create BigQuery tables, and change BigQuery schema of existing tables.

2.2.3. Common libraries or tools for building data pipelines

We have generic solutions for common use cases.

On Spark side, we have common Java classes for parsing EBCDIC files and loading data from relational databases.

On Airflow side, we have

- generic ETL framework for loading and transforming data using BigQuery
- common DAG creator for file ingestion
- common DAG for loading data from relation database to BigQuery
- common for creating views, data patching
- common DAG for streaming data from BigQuery to Kafka (via Spark)

These tools originated from different use cases in our work. They were not designed in a systematic way as a set of standardized tools of the data platform.

We need to revisit these tools from a platform perspective. And make sure we provide adequate tools or building blocks for outcome teams to build their pipelines. In ideal case, outcome teams should focus on use case specific logics. The platform specific details should be abstracted away from their work.

2.2.4. Standard ways of formatting numbers, dates, timestamps

Some data types can be presented differently with different format settings. Examples include numeric fields, date and timestamps fields. Particularly, some dates are stored as integer numbers in tables ingested from TSYS.

2.2.5. Standard metadata for all BigQuery tables

We need to define common metadata fields in all tables (e.g., record load time stamps, loader id, etc.).

2.3. Data with quality

2.3.1. quality check within data pipelines

Our existing data pipelines are primarily optimized for data transfer. To ensure data reliability and trustworthiness, we must strategically implement comprehensive data quality validation procedures throughout the pipeline lifecycle.

- duplicate data
- required fields
- critical checks based on business logics (e.g., account close date should not be earlier than account open date)

2.3.2. platform level quality monitoring via scans and reports

While data quality checks are critical, our current system lacks a platform-level data quality monitoring framework. This strategic gap hinders our ability to achieve a comprehensive overview of data health across the organization. Implementing platform-level monitoring will enhance our ability to detect and address data quality issues that may not be captured by individual pipeline checks, ensuring robust data integrity and reliability.

2.4. Automated and AI Powered (AI first assistant or automation)

2.4.1. data pipeline creation

The creation of new data pipelines currently involves numerous manual steps, including file layout and database analysis, Spark job and Airflow DAG development, and the provisioning of resources such as BigQuery datasets and tables, and Kafka topics and schemas. The absence of standardized guidelines and automation for these tasks leads to inefficiencies, increased potential for human error, and significant engineering effort.

To address these challenges, we will strategically invest in automated and AI-powered tools to streamline data pipeline development, reduce manual overhead, and improve data quality through consistent implementation and reduced human error.

> Pre-requisites of AI powered tools include:
>
> - sufficient and well-organized data
> - documented knowledge
> - tools with AI as first class users, human as second class users.
>
> We can start from cli AI tools.

2.4.2. regression testing

Regression testing within our data teams is currently a manual, time-consuming process. With around 1,000 DAGs in production—and that number continually increasing—any modification to shared code requires an immense amount of regression testing to ensure system stability and correctness across pipelines. The scale and complexity of this effort not only strains our resources but also increases the risk of missed issues.

By incorporating automated or AI-powered solutions, we can automate much of the heavy lifting involved in regression testing, enabling rapid detection of impacted workflows, intelligent prioritization of test cases, and continuous validation.

2.4.3. SQL performance tuning

Inefficient SQL queries can lead to prolonged wait times, unexpected spikes in cloud costs, and, in severe cases, even trigger organization-wide outages. Effective SQL performance tuning demands a thorough understanding of our data structures, query patterns, and underlying database technologies. Traditionally, optimizing SQL relies heavily on expert knowledge and manual analysis.

However, with advancements in AI, engineers now have access to intelligent tools that can automatically detect inefficiencies, recommend optimized query strategies, and proactively prevent performance bottlenecks. Leveraging AI-driven support for SQL performance tuning not only enhances system reliability and cost-efficiency, but also empowers our teams to focus on delivering greater business value.

2.5. Observable platform

2.5.1. performance and healthiness of each data pipelines

While the Composer dashboard provides valuable insights into the overall health of workers, schedulers, and task failure rates, it lacks visibility into the performance and status of individual data pipelines (DAGs). Implementing a dedicated dashboard for DAG-level monitoring is crucial, especially for operational batch processes that are subject to strict SLAs. Such granular monitoring enables us to promptly detect performance issues and SLA breaches at the pipeline level, ensuring more effective troubleshooting, higher reliability, and better accountability for our data workflows.

2.5.2. SLA of data products (or BQ tables)

Most of our data is loaded or updated on a regular schedule; however, the timing for each individual table can vary depending on the availability of source data. Furthermore, exceptions in the source systems or data pipelines may cause delays or changes to the usual schedule. Without clear visibility into whether data is up to date, data consumers are challenged to produce reliable results and make timely decisions.

Ensuring data freshness is essential for maintaining the integrity and usefulness of our data assets. From a data production standpoint, Service Level Agreements (SLAs) play a critical role by formally defining performance expectations and quality standards, including measurable metrics such as freshness, accuracy, and completeness. This agreement between data producers and consumers helps foster trust, accountability, and consistent delivery of high-quality data for business needs.

3. Work items

Category	To-do list
Standardization	• common libraries for building pipelines ◦ standardize current common utils ◦ move common utils to python library repositories and publish libraries to JFrog • common ways for create and evolve BigQuery tables

	• unify metadata in BigQuery tables • change spark jobs to use serverless dataproc • establish MLOps pipelines
Data quality	• Dataplex quality scan • Fix known issues (e.g., null values) • automated regression testing • enhance quality check within data pipelines
Security	• RBAC • BQ tokenization/masking
Lifecycle	• Overall view of data lifecycle (start to end) • Data retention & purging • Asset usage, how many copies
Discovery	• Data catalog • Data dictionary
Governance	• data contracts (BQ schema) • DGO reports/dashboards • define quality metrics and automate quality compliance check
Observability	• Provide state of data pipelines (to both engineering and business) • Collect key statistics (e.g., row counts, total amounts, and so on) • track data usage
AI support	• build AI tools or agents for creating data pipelines • add AI tests/reviews in the CI pipeline